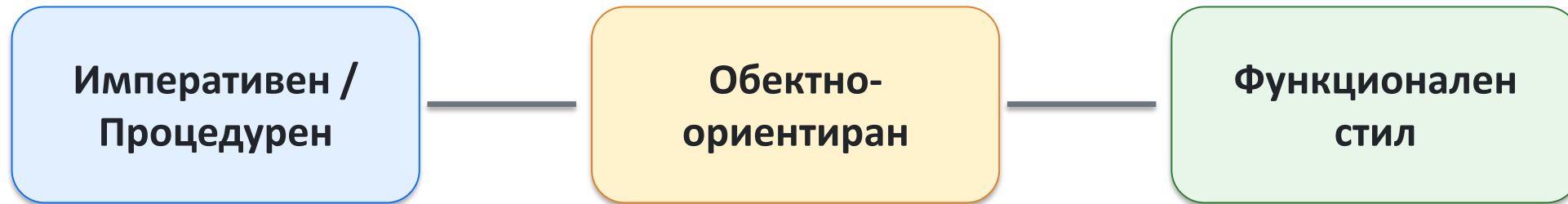


Съпоставка на парадигми в C#

Теоретичен час: императивен / процедурен, обектно-ориентиран и функционален стил

Една и съща задача • един език • различни начини на мислене и изпълнение



Цел на часа

Да сравним различни програмни стилове в рамките на един език – C#.

Да видим как една и съща задача изглежда в различен стил.

Да свържем познатите понятия `stack`, `heap` и `main thread` с изпълнението на кода.

Да подготвим основа за следващите уроци по функционално програмиране.

Какво е програмна парадигма?

Парадигмата е начин на организиране на кода и начин на мислене за решаване на задача.

В C# можем да използваме различни парадигми според проблема.

Един език
Много стилове

Една обща задача за всички примери

Да намерим сумата на четните числа в масива.

Общи входни данни

```
int[] numbers = { 2, 5, 8, 11, 14 };
```

Императивно
Как да го направя?

Процедурно
Разделям на методи

ООР
Обект с данни и
поведение

Функционално
Трансформации
върху данни

1. Императивен стил – всичко в Main

Пример

```
int[] numbers = { 2, 5, 8, 11, 14 };  
  
int sum = 0;  
  
for (int i = 0; i < numbers.Length; i++)  
{  
    if (numbers[i] % 2 == 0)  
    {  
        sum += numbers[i];  
    }  
}  
  
Console.WriteLine(sum);
```

Описваме подробно как да се достигне до резултата.

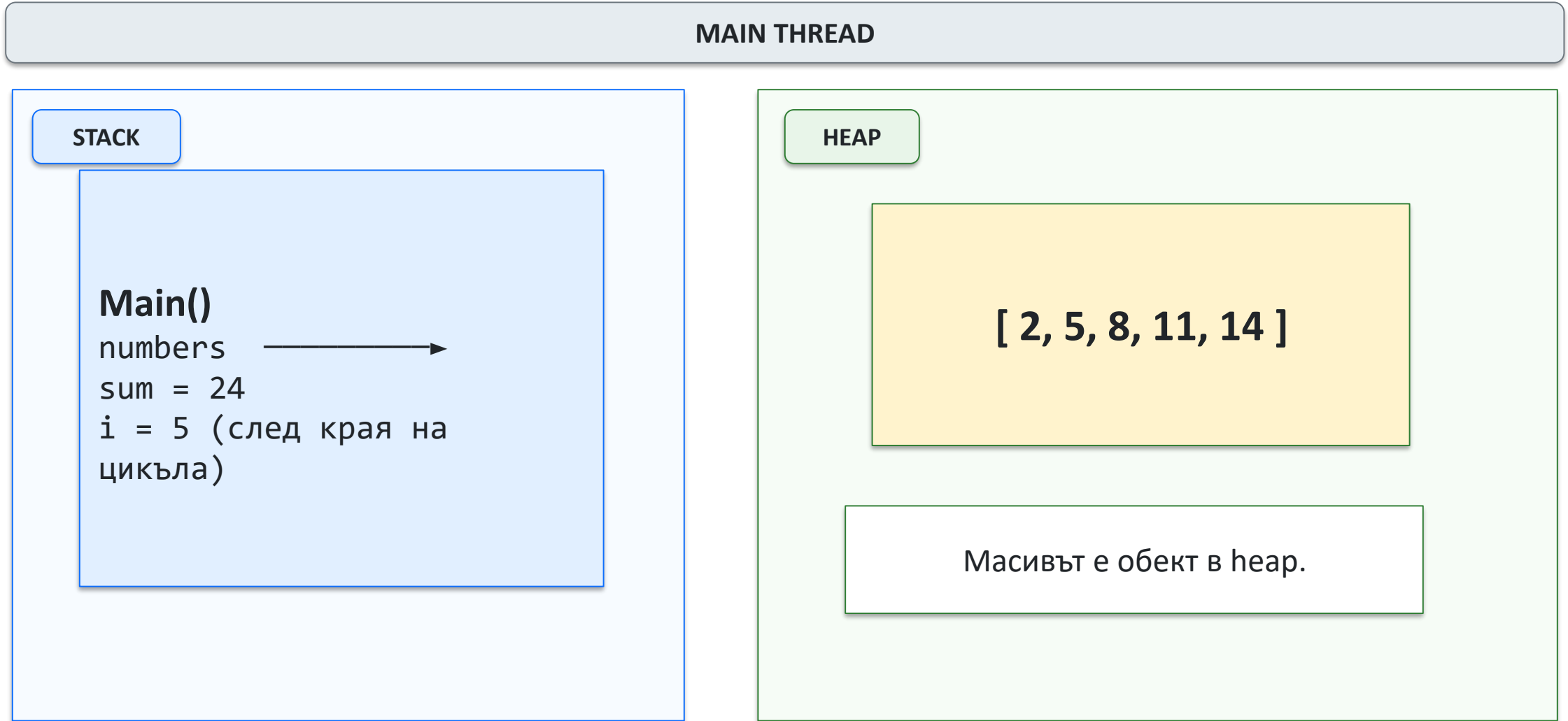
Използваме променливи, цикъл и условие.

Състоянието се променя стъпка по стъпка.

Изпълнението е последователно на main thread.

i: 0 -> 1-> 2->....->numbers.Length
sum: 0 → 2 → 10 → 24

Императивен стил – как изглежда в паметта



Учебна схема: локални променливи във *frame* на *Main*; масивът е референтен тип и стои в *heap*.

2. Процедурен стил – разделяме на методи

Пример

```
int[] numbers = { 2, 5, 8, 11, 14 };

int sum = SumEvenNumbers(numbers);
Console.WriteLine(sum);

static int SumEvenNumbers(int[] numbers)
{
    int sum = 0;

    foreach (int number in numbers)
    {
        if (number % 2 == 0)
        {
            sum += number;
        }
    }

    return sum;
}
```

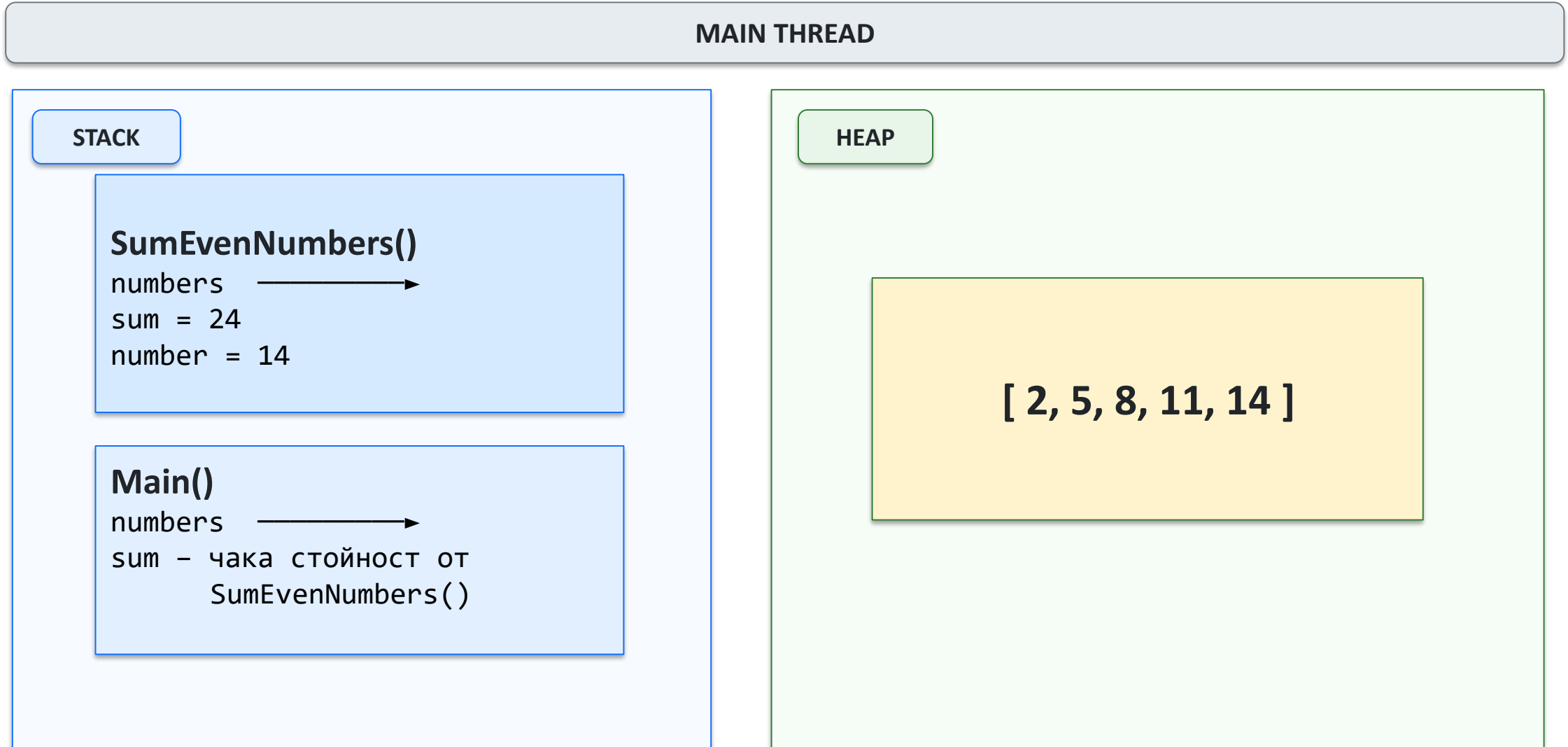
Алгоритъмът е почти същият, но е организиран в отделен метод.

Все още работим императивно – използваме променливи и цикъл.

Новото е, че при извикване на метод се появява нов stack frame.

Процедурното програмиране е разновидност на императивното.

Процедурен стил – stack frame при извикване на метод



При return frame-ът на `SumEvenNumbers()` се премахва, а frame-ът на `Main` остава.

3. Обектно-ориентиран стил – данни + поведение в обект

Клас

```
class NumberProcessor
{
    private int[] numbers;

    public NumberProcessor(int[] numbers)
    {
        this.numbers = numbers;
    }

    public int SumEvenNumbers()
    {
        int sum = 0;
        foreach (int number in numbers)
        {
            if (number % 2 == 0)
            {
                sum += number;
            }
        }
        return sum;
    }
}
```

Main()

```
int[] numbers = { 2, 5, 8, 11, 14 };
NumberProcessor processor = new
NumberProcessor(numbers);
int sum = processor.SumEvenNumbers();
```

Обектът пази състояние – полето numbers.

Обектът има поведение – метода SumEvenNumbers().

Данните и методите са събрани в една структура.

Обектно-ориентиран стил – обектът живее в heap

MAIN THREAD

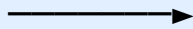
STACK

Main()

numbers

processor

sum = 24



HEAP

NumberProcessor

field: numbers →

method: SumEvenNumbers()

int[]

[2, 5, 8, 11, 14]

Обектът и масивът са в heap; Main държи референции към тях.

4. Функционален стил – операции върху данните

Пример

```
int[] numbers = { 2, 5, 8, 11, 14 };  
  
int sum = numbers  
    .Where(x => x % 2 == 0)  
    .Sum();  
  
Console.WriteLine(sum);
```

Мислим в трансформации

numbers

Where(четни)

Sum()

Не управляваме цикъла директно – описваме какви операции да се приложат върху данните. Lambda функцията `x => x % 2 == 0` се подава на `Where` като аргумент.

Функционален стил – изпълнение на Main thread

MAIN THREAD

STACK

lambda(x)
пример: x = 8

LINQ iterator / Where -> MoveNext()

Enumerable.Sum()
Accumulator

Main()
numbers, sum

HEAP

[2, 5, 8, 11, 14]

LINQ iterator
source
predicate
state / position
current

Func<int,bool>

Съпоставка на стиловете

Стил	Как мислим	Състояние	Какво виждаме в паметта
Императивен	Как да изпълня стъпките?	Променя се често	Frame на Main + масив в heap
Процедурен	Как да разделя задачата на методи?	Локално в методите	Повече stack frames
ООР	Кой обект пази данни и поведение?	Състояние на обекта	Обекти в heap + refs в stack
Функционален	Какви трансформации да приложа?	Стремим се да го ограничим	LINQ pipeline + lambda на main thread

Извод: C# е многопарадигмен език – в реалните програми често смесваме стилове според проблема.

Обобщение и въпроси за дискусия

Различните парадигми не са „конкуренти“ – те предлагат различни гледни точки.

Императивният и процедурният стил са близки и работят със стъпки и състояние.

OOP организира кода около обекти.

Функционалният стил ни помага да мислим в трансформации и композиция.

Въпрос 1

Кой стил разпознавате най-лесно в кода?

Въпрос 2

Къде се намира масивът и защо?

Въпрос 3

Създава ли `lambda` нова нишка?

Въпрос 4

Къде се появява нов `stack frame`?